

Securing Blockchain-based E-Voting through Shamir's Secret Sharing over Ethereum

Esma Beydili¹, Umut Can Cabuk², Gokhan Dalkilic¹, Yusuf Ozturk²

¹*Dept. of Computer Engineering, Dokuz Eylul University, Izmir, Turkey*

²*Dept. of Electrical and Computer Engineering, San Diego State University, San Diego, CA, USA*
esma beydili18a@gmail.com, ucabuk@sdsu.edu, gokhan.dalkilic@deu.edu.tr, yozturk@sdsu.edu

Abstract—Smart contracts are one of the main drivers of advances in blockchain technology and its applications, like e-voting. This study proposes the integration of Shamir's Secret Sharing (SSS) algorithm into blockchain-based electronic voting (e-voting) systems powered by smart contracts to enhance voter privacy without compromising integrity. We developed a prototype on Ethereum using Remix IDE and validated the design over a test network using test accounts. We evaluated this integration based on cost, complexity, privacy, and accessibility metrics. We examined existing privacy-enhancing solutions and focused on their application within e-voting systems. Our analysis considers gas fees on Ethereum as well as the computational and data storage overheads caused by SSS. Although the current implementation of secret sharing falls short of preserving privacy (due to inherent transparency of smart contracts), it shows that it can successfully obfuscate the candidate selection of voters. Accessibility is measured through the time required for system access and vote casting. Overall, our results indicate that while integrating SSS increases cost and system complexity, it holds significant potential for enhancing privacy in e-voting systems.

Index Terms—Blockchain, Ethereum, e-voting, Shamir's Secret Sharing, smart contract, Solidity.

I. INTRODUCTION

Smart contracts represent self-executing agreements codified on a blockchain, where predetermined conditions are embedded in code and automatically enforced upon satisfaction. Much like traditional legal contracts, these digital contracts are agreed upon by the involved parties, yet they obviate the need for human intervention once deployed. Their transparent nature—where every transaction is permanently recorded on an immutable ledger—has led to widespread adoption, particularly within the financial sector and on platforms such as Ethereum [1]. The primary reason for the preference for smart contracts is their transparency, meaning they can be viewed and verified by anyone. Once a smart contract is written to the blockchain, it becomes immutable as well. Since the transactions are permanently recorded, these records can be trusted in case of disputes.

Electronic voting (e-voting) systems have also benefited from smart contract technology. In e-voting, voters cast their ballots electronically via devices such as computers or smartphones, and the results are automatically computed and announced without intermediaries. This automation expedites the

voting process and enhances efficiency compared to traditional methods. The decentralization, security, and immutability of blockchain-based systems are highly desirable, but the inherent transparency of smart contracts can conflict with the essential requirement for voter privacy, such as the association between voters and the candidates they voted for [2], [3].

To address this dichotomy, this paper explores the integration of secret-sharing techniques into smart contract-based e-voting systems. The primary objective is to introduce a controlled human oversight mechanism into critical transactions, such that sensitive information remains protected until a predetermined quorum is reached. Secret sharing enables a secret to be divided into multiple fragments that are individually unintelligible, ensuring that only when a sufficient number of pieces are combined can the original secret be reconstructed. This approach not only mitigates the privacy issues introduced by smart contract transparency but also prevents major errors by requiring multi-party authorization. The foundation of secret sharing lies in well-established cryptographic methods. Shamir's Secret Sharing (SSS) scheme [4] and Blakley's method [5] provide robust frameworks for distributing a secret among multiple parties. These schemes are underpinned by the concept of perfect secrecy as formalized in Shannon's one-time pad [6], which employs randomness and modular arithmetic to ensure that individual fragments reveal no information about the complete secret.

In smart contracts, the number of participants required to reconstruct a secret can be adjusted to balance security with fault tolerance. As the number of involved parties increases, maintaining confidentiality becomes more challenging, particularly if a subset of the secret pieces is lost or compromised. This paper assesses the technical complexity, cost, privacy, and accessibility of integrating secret sharing within smart contract systems. Furthermore, it reviews current applications of these techniques in blockchain wallets [7] and voting systems [8], [9], while also identifying scenarios where secret sharing remains underutilized. Through SSS, this work aims to develop robust, privacy-preserving, and efficient e-voting systems that harness the strengths of decentralized architectures while safeguarding sensitive voter information.

The rest of the paper is organized as follows: Section 2 reviews related works, Section 3 describes the proposed system, Section 4 presents the tests, validation, results, and discussion, and Section 5 concludes the study.

TABLE I
COMPARISON OF SOME OF THE EXISTING APPROACHES

Method	Auditability	Anonymity	Authentication & Privacy	Reliability	Scalability	Verifiability
Sudi [10]	+	Partly	Partly (SSS)	+ (Ethereum, MetaMask)	-	+
Awalu et al. [11]	+	Partly	Partly (SMS, Unique ID)	+	+	+
Tarasov et al. [12]	+	+ (ZCash)	+	-	N/A	+
Lui and Wang [13]	+	Partly	Partly ((Blind signature))	N/A	N/A	+
Sun et al. [14]	N/A	+	+ (Quantum Key Dist.)	N/A	N/A	+
Fusco et al. [15]	N/A	+	+ (SSS)	+	N/A	N/A
Bartolucci et al. [8]	N/A	+	+ (CircleShuffle, SSS)	N/A	-	+
Pramulia et al. [16]	+	Partly	-	+ (Ethereum, MetaMask)	N/A	+
Khoury et al. [17]	N/A	-	+ (SMS, biometric auth.)	+ (Ethereum)	N/A	+
James et al. [18]	+	+	+ (Fingerprint)	+	+	+
Sharma et al. [19]	N/A	-	+	-	+	+

II. RELATED WORKS

Recent studies have explored various decentralized architectures for secure data sharing and e-voting systems by leveraging blockchain technology and advanced cryptography [9], [18]–[20]. Naz et al. [21] proposed a decentralized framework that integrates InterPlanetary File System (IPFS) with blockchain to facilitate secure data sharing and digital asset delivery. In their approach, the data owner fragments metadata into several secret pieces using Shamir’s Secret Sharing (SSS) algorithm [4] and enforces access control through smart contracts on the Ethereum [1] blockchain platform. Each user first verifies their identity to obtain access to the data in accordance with the access rules set by the data owner. User authentication is achieved via RSA signatures, ensuring transparency, confidentiality, and owner verifiability. The study further evaluates system performance in terms of “gas” consumption and associated fees, comparing the efficiency of SSS with that of conventional encryption standards such as AES-128/256.

Tran et al. [7] emphasize the potential of a distributed key architecture for blockchain wallets. Their framework divides a user’s cryptographic key into three components: one managed by network nodes via Distributed Key Generation (DKG), one stored locally on the client, and a backup retained by the user. Although the decentralization of key management and use of SSS improves security and responsiveness, the study also highlights vulnerabilities such as node fraud and system downtime, which may impact overall system robustness. Zhang et al. [22] introduce a Fair Hierarchical Secret Sharing Scheme designed to address scenarios in which participants possess varying levels of authority. By distributing secret shares across multiple hierarchical layers and utilizing smart contracts to manage obligations, the proposed scheme improves the efficiency and fairness of secret reconstruction while eliminating the need for a trusted third party.

In the context of blockchain-based voting systems, Bartolucci et al. [8] introduce a protocol that combines Shamir’s Secret Sharing with the Circle Shuffle mechanism. While SSS facilitates the secure partitioning and reconstruction of vote data under predefined conditions, the Circle Shuffle protocol anonymizes the association between voters and ballots, thereby enhancing vote privacy. However, the protocol does not fully mitigate the risk of duplicate or malformed votes,

which may compromise the integrity of the voting process. Pramulia et al. [16] developed an Ethereum-based e-voting system that delegates voter authentication to MetaMask and tested it on the Ropsten testnet. Their system aspires to adhere to six fundamental electoral principles by reducing cost and expediting result announcement compared to traditional voting methods. Nonetheless, the transparency inherent to smart contracts allows for reverse engineering of vote data, compromising voter privacy. The authors suggest that integrating biometric authentication could provide a more robust identity verification mechanism, as also discussed in [20]. Likewise, Khoury et al. [17] propose an Ethereum-based e-voting system that incorporates mobile phone authentication through a dedicated web application. The study advocates for the integration of fingerprint recognition or specialized devices to replace traditional SMS-based verification systems, thereby enhancing security and usability. Awalu et al. [11] propose a permissioned blockchain-based model to guarantee system accuracy. Tarasov and Tewari [12] present an anonymous model based on Ethereum. For privacy protection purposes, Sun et al. [14] follow the Quantum Blockchain approach. Yavuz et al. [23] developed a scalable running prototype based on Ethereum, yet it lacked privacy. Fusco et al. [15] partly addressed privacy by their two-chain model. Lui and Wang [13] are shown to provide verifiability with their work using blind signatures. In addition to those studies, there are also notable mentions among industry players and governments: Voatz Mobile Elections Platform (available: <https://voatz.com> Accessed: 07-Mar-2025) is the US’s system for overseas military voters that uses biometric verification such as fingerprints or retina scans. Kaspersky Polys Online Voting System (available: <https://box.kaspersky.com/f/e68a161d8e7241909ea3> Accessed: 07-Mar-2025) is an e-voting application relying on blockchain and other transparent cryptographic methods. Some of the existing approaches were analyzed and compared in Table I, where + and - indicate explicit existence or absence of a feature, and N/A indicates no mention of it.

The Votereum project [24] presents an e-voting system deployed on the Rinkeby test network that aims to deliver a fair, universally verifiable electoral process while satisfying key requirements such as privacy, uniqueness, and verifiability. Despite its innovative design, the system remains susceptible to potential sabotage by authorities, and more importantly, the application server and the online ballot regulator introduce

significant centralism to the system architecture, which is somewhat the opposite of the expectations from a blockchain-based e-voting system. Harris [25] investigates the integration of Zero-Knowledge Proofs (ZKP) with Shamir's Secret Sharing to enhance the reliability and privacy of smart contracts in voting scenarios, particularly within the financial sector. Two configurations are examined—one applying ZKP prior to SSS and the other using the output of SSS as the input for ZKP—with simulation results providing comparative insights into their operational efficiency.

III. THE PROPOSED SYSTEM

To tackle the voter privacy problem of blockchain-based e-voting systems simply and without sacrificing decentralization, we propose a novel blockchain-based e-voting system that leverages SSS to safeguard voter privacy while ensuring transparency, security, and decentralization. The proposed system is a full-scale application implemented through smart contracts within the Ethereum network. It was mainly based on [9] and [10], yet the guidelines we assumed for the voting scheme are as follows (mainly adopted from [10]):

- The contract owner (i.e., election organizer) can input one or more options for people to vote on (i.e., candidates).
- The contract owner can set start and end times for voting.
- A voter can vote for any option defined by the contract owner during the voting period.
- A voter can vote only once during the voting period.
- The smart contract can return vote counts for each option.

This voting system, completely developed in Solidity language, also aims to integrate features like counting the number of voters, setting a minimum number of participants needed to declare the result (i.e., threshold), and integrating Shamir's Secret Sharing (SSS) algorithm to share the secret. Smart contracts written in Solidity run on the Ethereum Virtual Machine and are portable to different networks. We utilized Remix Ethereum IDE (<https://remix.ethereum.org>, accessed: Feb.25,2025) to develop, deploy, and test the system. It is an open-source integrated development environment accessible online, allowing the use of local and test networks. Fig. 1 presents the user interface.

To give context, Shamir's Secret Sharing algorithm [4], is a polynomial used to share a secret value, and this polynomial can often be visualized on a 2-dimensional plane. The results of the function $f(x)$ for various x values on the polynomial correspond to the shares. Each share $(x, f(x))$ is considered a point. These points can be thought of as coordinates containing the information needed to reconstruct the secret. Each of these shares will be distributed to the voters. The smart contract build for the proposed system is designed to generate such a polynomial with corresponding coefficients.

Our improvements to Sudi's voting system [10] include the adaptation of SSS and other features we considered into the smart contract through dedicated functions. These functions are critical to ensure the confidentiality of votes and to obtain the result only if a certain number of voters provide the correct

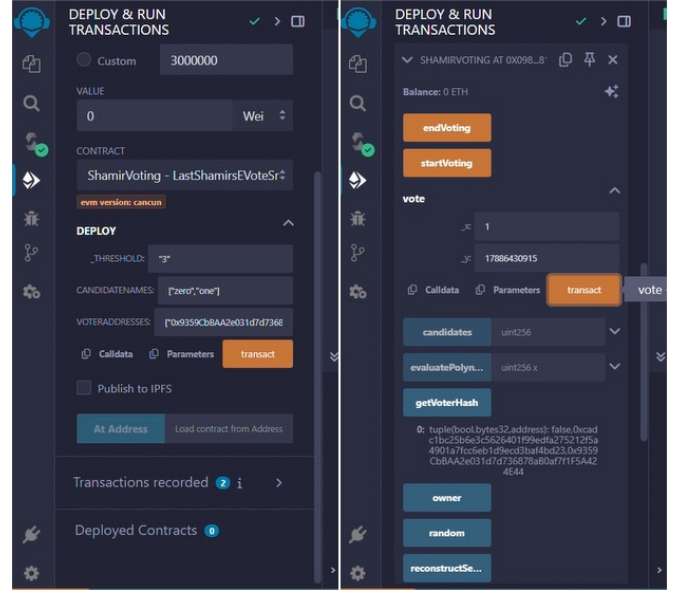


Fig. 1. User interface of Remix IDE: deployment panel on the left and execution panel on the right.

shares. The main functions added to the smart contract are:

- **generateSecretAndShares**: Generates a random secret and the polynomial that shares it. It also assigns a share to each participant.
- **evaluatePolynomial**: Evaluates the polynomial for a given x value and calculates the secret's share.
- **vote**: Checks whether the votes were valid after participants voted. The validity condition is the participants' use of "the correct shares".
- **reconstructSecret**: Retrieves the secret using Lagrange interpolation. Correctly restoring the secret requires participants to provide their shares.
- **endVoting**: Checks if the winner was announced and whether the secret was correctly reconstructed, at the end of the voting. Returns the winner as the result.

The vote function used by voters during the election:

```
function vote(uint256 _x, uint256 _y) public onlyVoter {
    require(votingStarted, "Voting not started");
    require(!votersmsg.sender.hasVoted, "Already voted");
    bool correctVoting = false;
    uint256 candidateIndex = 0;
    for(uint256 i = 0; i < candidates.length; i++) {
        if(_y - i == votersmsg.sender.share.y) {
            candidateIndex = i;
            _y = _y - i;
            correctVoting = true;
            break;
        }
    }
    require(correctVoting, "Invalid share");
    Voter storage voter = votersmsg.sender;
    require(voter.share.x == _x & voter.share.y == _y,
        "Invalid share");
    shares.push(Share({x: _x, y: _y}));
    votersmsg.sender.hasVoted = true;
    votersHashmsg.sender.hasVoted = true;
    candidates[candidateIndex].voteCount += 1;
    totalVotes += 1;
}
```

The secret reconstruction:

```

function reconstructSecret() public view returns (uint256) {
    require(shares.length > 0, "Shares list is empty");
    (int256 result = 0;
    for ((uint256 i = 0; i < shares.length; i++) {
        (int256 numerator = 1;
        (int256 denominator = 1;
        for ((uint256 j = 0; j < shares.length; j++)
            if (i != j) {
                require(shares[i].x != shares[j].x,
                    "Error: x values must be unique!");
                numerator = numerator *
                    (0 - (int256(shares[j].x)));
                denominator = denominator *
                    ((int256(shares[i].x) -
                    (int256(shares[j].x)));
            }
        require(denominator != 0,
            "Error: Denominator is zero!");
        (int256 lagrangeCoefficient = numerator /
            denominator;
        result += (int256(shares[i].y) *
            lagrangeCoefficient;
    }
    return (uint256(result);
}

```

Some conditions for a successful voting session:

```

require(votingStarted, "Voting not started");
require(totalVotes >= threshold, "Not enough votes");
require(reconstructedSecret == secret,
    "Secret reconstruction failed");

```

IV. TESTS AND DISCUSSION

The development was followed by a comprehensive validation, algorithmic analysis, and performance measurements through a number of tests performed after three scenarios. We utilized Sepolia, an Ethereum-based test network, as our test environment, where we deployed our Solidity smart contracts and executed our functions (through Remix IDE). The MetaMask wallet was used to connect to the Ethereum network and perform transactions on the test network. Sepolia provided a secure environment for tests and served as a platform to check whether the smart contracts functioned as intended. Furthermore, the entire codebase of the project is shared publicly on GitHub [26].

The "gas fee" unique to Ethereum transactions (incl. smart contracts) is considered for cost measurement. Each operation to be recorded on the ledger, such as sending tokens, updating data, or running smart contracts, incurs a computational cost. yet, "Gas" is the unit used to measure the processing power required for such a transaction. Adopting SSS into an e-voting contract increases transaction costs, as additional computation and data processing are needed to share and reconstruct the secret. To calculate that increase, it is reasonable to compare the gas consumption of the baseline system against our proposal integrating the secret sharing. While gas fees disincentivize spam or malicious transactions, the actual amount varies per the demand on the Ethereum network as $\text{Gas Fee} = \text{Gas Consumed} \cdot \text{Gas Price}$, where the latter fluctuates.

A. Test Scenarios

For validation purposes, we designed three scenarios that follow the same main flow, as shown in Fig. 2, but differ in their specific nuances.

1) *Scenario 1*: The voting involves two candidates: "Shamir" and "Blakely". A total of three accounts (account1, account2, and account3) participate as voters. The threshold value for the secret sharing is set equal to the number of voters, meaning that all voters must vote to reveal the final result. The voting proceeds as follows: three participants cast their votes correctly, and three votes are directed to "Blakely" (index 1). The expected result is that "Blakely" will receive all the votes, while the actual results follow the expected outcome. The system will display the results, including the breakdown of votes and the time taken for the entire process to complete. Additionally, details on the secret shares, the distributed shares, and the gas fees consumed will be reported. When calculating the gas fee, the fee accrued in the contract setup, voting initiation, voting process of all users, and voting termination were taken into account. Records were accessed via Metamask. Table II summarizes the scenario.

2) *Scenario 2*: The voting process is initialized with two candidates: "Crypto" and "Netsec." A total of three participants (account1, account2, and account3) are expected to vote. The threshold for secret reconstruction is set to the number of voters. All participants intend to cast their votes for the candidate "Crypto" (index 0). However, during the voting phase, account1 attempts to cast their vote using an incorrect secret share, whereas account2 and account3 vote correctly. After the voting concludes, the results are evaluated. Yet, the voting result cannot be revealed due to the invalid secret share from account1. An error message is displayed instead. A subsequent resubmission with the correct share may or may not be offered to account1. The secret must be retrieved for the "endVoting" function to return a result. Here the secret

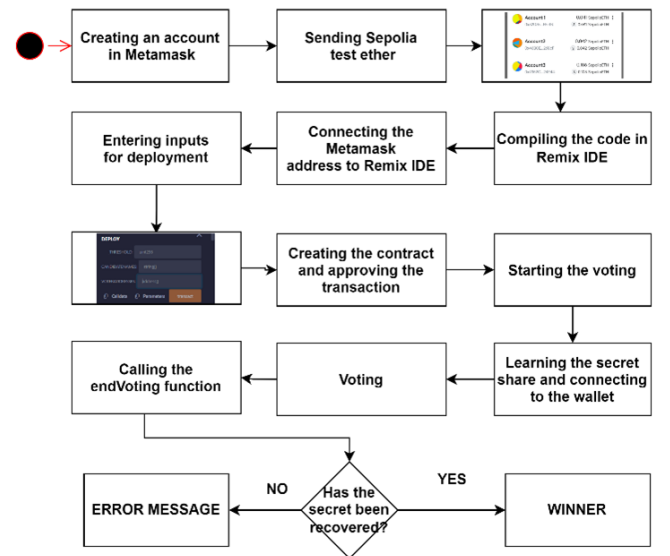


Fig. 2. Execution stages of e-voting test scenarios.

TABLE II
TEST SCENARIO 1

Candidates	Voter Count	Vote Distribution	Expected Result	Actual Result	Incorrect Input	Gas Fee
Shamir (0) Blakely (1)	3	0 for Shamir 3 for Blakely	Blakely wins w/ unanimous votes	Blakely wins w/ unanimous votes	No	0.004409 Sepolia ETH

TABLE III
TEST SCENARIO 2

Candidates	Voter Count	Vote Distribution	Expected Result	Actual Result	Incorrect Input	Gas Fee
Crypto (0) Netsec (1)	3	0 for Crypto 3 for Netsec	ERROR	ERROR (Non-termination of the contract)	Yes	0.004300 Sepolia ETH

TABLE IV
TEST SCENARIO 3

Candidates	Voter Count	Vote Distribution	Expected Result	Actual Result	Incorrect Input	Gas Fee
Umut (0) Esma (1)	3	1 for Umut 2 for Esma 1 for invalid index (3)	"One" wins w/ majority vote	ERROR Then Esma wins w/ majority votes	Yes	0.006230 Sepolia ETH

could not be recreated due to an incorrect secret part.. Table III summarizes the scenario.

3) *Scenario 3*: Evaluates the behavior of the voting system when a voter provides an invalid input. The voting process starts with two candidates: "Umut" and "Esma". Again, there are three participants (account1, account2, and account3), and the threshold is the number of voters. All Voters first compare the hash of secret pieces y and continue if it is correct. In the voting phase, account3 tries to vote to a non-existent index (index 2) using index $y + 2$; meanwhile, account1 and account3 cast their votes correctly. Two valid votes are cast for candidate "Esma" (index 1). After the voting is over, the results are evaluated. Since the results cannot be determined due to the invalid vote from account3, the problem is later solved by letting account3 to change their vote to the valid candidate "Umut" (index 0). The implemented code does not actually allow the voter to vote for a wrong index number. Table IV summarizes the scenario. It reflects the later-changed vote of account3 in the vote distribution column. If two voters vote using the same secret pieces, an "invalid shares" error is returned and the transaction is not completed.

B. Computational Complexity and Scalability

Integrating Shamir's Secret Sharing increases the technical complexity of e-voting as each sharing of the secret needs to be processed separately. Indeed, the complexity of the code is impacted by three main phases: initialization, voting, and finalization. The initialization process covers the secret generation, coefficient determination, and share distribution operations. The complexity is found to be $O(v \cdot t)$, where v is the number of voters, and t is the threshold used for secret sharing. The voting procedure is run by each voter. The computational complexity for a single voter is $O(1)$, and the total complexity is $O(v)$. During the finalization, the secret is reconstructed using the Lagrange interpolation method, and the winning candidate is determined. The complexity of secret

reconstruction is $O(s^2)$, where s is the number of secret shares used. Finding the winner among all candidates has a complexity of $O(c)$, where c is the number of candidates.

The most complex operation in the entire program is the Lagrange interpolation in the finalization process, with a complexity of $O(s^2)$. Yet, this operation is performed only in the last stage and involves a limited number of secret shares, so in practice, it should not significantly impact the overall performance. As v and t increase, the polynomial computations in the initialization process scale with $O(v \cdot t)$. The individual validation operations in the voting process are $O(1)$, ensuring scalability. The load in the finalization stage remains manageable as long as $s \leq v$. Yet, the overall computational complexity of the program can be found as $O(v \cdot t + s^2 + c)$. While practical implications depend on v , t , s , and c , the overall performance is primarily dominated by the $v \cdot t$ term. The ballot contract in [10] has a simpler voting mechanism and, thus, a lower complexity of $O(v + c)$, but it lacks our privacy and security features.

C. Accessibility for Voters

To assess the accessibility of the new system, a voter's access time before the integration of Shamir's algorithm into the voting was compared against the new development with SSS (per a concluded voting). The tests were carried out at time frames close to each other when the gas price was lower. Since this was a testing process, considering that a single person did the same operations for all candidates, both cases were spent above expectations but still similar. However, it was observed that, for the version using secret sharing, a tradeoff should be considered between cost and time due to the algorithmic complexity of the voting termination process, especially for hours when the gas price is high.

We evaluated privacy, considering whether any central node can access all the election data. We also investigated whether the secret's shares can be linked to a user's identity and if user anonymity is preserved. Secret sharing was adopted to ensure voter authorization so that only the accounts (i.e. addresses) who are given their own shares can vote by entering the secret pieces given to them while casting their votes. The same secret piece was prevented from being entered by two accounts, even if it is valid. A hash comparison to see a voter's voting status was avoided for privacy reasons. In the test phase, sending secret shares to voters was not done using IPFS or HCE; therefore, this case was not tested. The candidate for whom a vote was cast was hidden by a simple adjustment to the voting function. First, the candidate index was extracted from the parameters submitted as input. Then, the y index corresponding to the voter's x index is computed. When this y index is subtracted from the voter's input, if the remaining number is equal to a valid candidate index, the vote is accepted and counted towards that index.

To increase privacy, the secret pieces must first be shared secretly prior to the election, which was left out of the scope of this study. Yet, the hash of each y index associated with each secret piece is kept within the smart contract and is visible to nodes. So that any voter can verify the integrity and authenticity of the vote piece sent to them by comparing the calculated hash of the sent item against the hash hosted by the smart contract through a checker function. Hence, protection against men-in-the-middle and replay attacks is achieved.

V. CONCLUSION

This study proposes a blockchain-based e-voting system that integrates Shamir's Secret Sharing (SSS) algorithm through a customized Ethereum smart contract, builds a prototype of the proposed using Solidity, and runs tests to evaluate some key features like cost, algorithmic complexity, privacy, and accessibility. The integration of SSS drastically improved ballot anonymity and voter privacy thanks to the mandatory use of pre-shared "secret shares" by voters as another input in addition to the vote itself. However, the time complexity of the voting process was increased as well. It can be expressed as $O(v \cdot t + s^2 + c)$, where v is the number of voters, t is the voter turnout threshold, and s is the number of used secret shares. There is also an increase in gas fees, especially during network congestion. The integration created some mild difficulties in terms of accessibility as the procedure became slightly more time-consuming. Despite those, we see great potential in the integration of SSS into critical blockchain-based e-voting systems that aim to imitate or replace traditional elections.

Future work will focus on developing methods to securely distribute secret shares so that no single central entity can fully access them. Research may also explore the use of alternative channels, such as SMS-based secret sharing, to obscure the vote index and further enhance voter privacy.

- [1] Vitalik B. A next generation smart contract & decentralized application platform. 2015.
- [2] Morgen E. P. Blockchain world - do you need a blockchain? this chart will tell you if the technology can solve your problem. *IEEE Spectrum*, 54:38–60, 2017.
- [3] Y. Zhao, Y. Li, Q. Mu, B. Yang, and Y. Yu. Secure pub-sub: Blockchain-based fair payment with reputation for reliable cyber physical systems. *IEEE Access*, 6:12295–12303, 2018.
- [4] A. Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, November 1979.
- [5] G. R. Blakley. Safeguarding cryptographic keys. In *Managing Requirements Knowledge, Int. Wkshp. on*, page 313, Los Alamitos, CA, USA, June 1979. IEEE Computer Society.
- [6] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.
- [7] T. Tran-Ngo, T. Nguyen, K. V. Tan, M. Nguyen-Thi, T. Nguyen-Hoang, T. Nguyen, and N. Dinh. Distributed key architecture for blockchain wallets. In *16th Int. Conf. on Comput. and Automation Engineering (ICCAE)*, pages 93–97, 2024.
- [8] S. Bartolucci, P. Bernat, and D. Joseph. Sharvot: secret share-based voting on the blockchain. In *1st Int. Wkshp. on Emerging Trends in Software Engineering for Blockchain, WETSEB '18*, page 30–34, New York, NY, USA, 2018. Assoc. for Computing Machinery.
- [9] U. C. Cabuk, E. Adiguzel, and E. Karaarslan. A survey on feasibility and suitability of blockchain techniques for the e-voting systems. *IJARCCCE*, 7(3):124–134, March 2018.
- [10] A. Sudi. Voting system smart contract. GitHub Repository, github.com/andresudi/Voting-Smart-Contract (accessed: Feb. 26 2025), 2021.
- [11] I. L. Awalu, P. H. Kook, and J. S. Lim. Development of a distributed blockchain evoting system. In *10th Int. Conf. on E-Business, Management and Economics, ICEME '19*, page 207–216, New York, NY, USA, 2019. Assoc. for Computing Machinery.
- [12] P. Tarasov and H. Tewari. The future of e-voting. *IADIS Int. J. Comput. Sci. Inf. Syst.*, 12(2):148–165, 2017.
- [13] Y. Liu and Q. Wang. An e-voting protocol based on blockchain. *IACR Cryptol. ePrint Arch.*, 2017:1043, 2017.
- [14] X. Sun, Q. Wang, P. Kulicki, and M. Sopek. A simple voting protocol on quantum blockchain. *Int. J. of Theoret. Phys.*, 58(1):275–281, 2018.
- [15] F. Fusco, M. I. Lunesu, F. E. Pani, and A. Pinna. Crypto-voting, a blockchain based e-voting system. In *10th Int. Joint Conf. on Knowledge Discov., Knowledge Eng. and Knowledge Mngmnt. SCITEPRESS*, 2018.
- [16] D. Pramulia and B. Anggorojati. Implementation and evaluation of blockchain based e-voting system with ethereum and metamask. In *Int. Conf. on Informatics, Multimedia, Cyber and Inform. System (ICIMCIS)*, pages 18–23, 2020.
- [17] D. Khoury, E. F. Kfoury, A. Kassem, and H. Harb. Decentralized voting platform based on ethereum blockchain. In *IEEE Int. Multidisciplinary Conf. on Engineering Technol. (IMCET)*, pages 1–6, 2018.
- [18] Vivekanandan G, P G James T, Divya B, Madhav V Tejas, Naveen K T K, and Yuvanesh P. Votchain: Promising a secure and transparent election using blockchain and biometrics. In *Int. Conf. on Power, Energy, Control and Transmission Syst. (ICPECTS)*, pages 1–6, 2024.
- [19] P. Sharma, A. Gupta, A. D. Pandey, A. S. Negi, K. V. Mishra, and R. Garg. E-voting based upon blockchain system. In *Int. Conf. on Emerging Innov. and Advan. Computing (INNOCOMP)*, pages 157–163, 2024.
- [20] U. C. Cabuk, T. Şenocak, and E. Demir. A proposal on initial remote user enrollment for ivr-based voice authentication systems. *IJARCCCE*, 6(7):118–123, July 2017.
- [21] M. Naz, F. A. Al-zahrani, R. Khalid, N. Javaid, A. M. Qamar, M. K. Afzal, and M. Shafiq. A secure data sharing platform using blockchain and interplanetary file system. *Sustainability*, 11(24), 2019.
- [22] En Zhang, Ming Li, Siu-Ming Yiu, Jiao Du, Jun-Zhe Zhu, and Gang-Gang Jin. Fair hierarchical secret sharing scheme based on smart contract. *Information Sciences*, 546:166–176, 2021.
- [23] E. Yavuz, A. Koc, U. C. Cabuk, and G. Dalkilic. Towards secure e-voting using ethereum blockchain. *2018 6th Int. Symp. on Digital Forensic and Secur. (ISDFS)*, pages 1–7, 2018.
- [24] L. V. Thuy, K. Cao-Minh, C. Dang-Le-Bao, and T. A. Nguyen. Votereum: An ethereum-based e-voting system. In *IEEE-RIVF Int. Conf. on Computing and Commun. Technol. (RIVF)*, pages 1–6, 2019.
- [25] C. G. Harris. Consensus-based secret sharing in blockchain smart contracts. In *2019 Int. Wkshp. on Big Data and Inform. Secur. (IBWIS)*, pages 79–84, 2019.
- [26] E. Beydili. Voting system smart contract. GitHub Repository, github.com/Esma222/EvotWithShamir (accessed: Feb. 26 2025), 2025.